Laravel + Electron.js Desktop ERP

Complete implementation guide for an existing Laravel project — Ubuntu development, remote MySQL, secure Electron shell, and Windows EXE packaging.

Recommended architecture

```
Development
Ubuntu
  ■■■ Laravel ERP: /var/www/html/M-ERP-Desktop
  ■      ■■■ PHP + Composer
  ■      ■■■ Blade / Bootstrap / jQuery / Select2 / Tabulator
  ■      ■■■ Remote MySQL
  ■
  ■■■ Electron: /var/www/html/M-ERP-Desktop-electron
         ■■■ main.js
         ■■■ preload.js
         ■■■ package.json

Production
Windows PC
   ↓
M-ERP Desktop.exe
   ↓
Electron
   ↓ HTTPS
Laravel Server
   ↓
Remote MySQL
```

> Key principle: Electron is the desktop shell. Your Laravel application remains the application/backend. Do not make Electron connect directly to MySQL.

1. Prerequisites

Requirement	Purpose	Check command
Node.js + npm	Run Electron and package the desktop app	node -v / npm -v
PHP	Run Laravel locally	php -v
Composer	Install Laravel dependencies	composer -V
Laravel ERP	Existing application	php artisan --version
Remote MySQL	Production database	Configured in Laravel .env

2. Keep Electron separate from Laravel

```
/var/www/html/
■■■ M-ERP-Desktop/
■   ■■■ Laravel project
■■■ M-ERP-Desktop-electron/
    ■■■ Electron project
```

This separation keeps Composer/PHP dependencies independent from npm/Electron dependencies and allows the Laravel and desktop applications to be released independently.

3. Prepare Laravel first

```
cd /var/www/html/M-ERP-Desktop
composer install
php artisan key:generate
php artisan serve
```

Open **http://127.0.0.1:8000** in a browser and confirm that the complete ERP works before debugging Electron.

If Composer reports that composer.json and composer.lock are out of sync, fix the lock file first. For example, if doctrine/dbal is intentionally required:

```
grep -n "doctrine/dbal" composer.json
composer update doctrine/dbal
composer install
```

> Do not use **compose install**; that is not a Composer command. Do not run a broad composer update unless you actually need to update all dependencies.

4. Create the Electron project

```
cd /var/www/html
mkdir M-ERP-Desktop-electron
cd M-ERP-Desktop-electron

npm init -y
npm install --save-dev electron

npx electron --version
```

If Ubuntu shows the Chromium SUID sandbox error, fix only the sandbox helper rather than disabling the sandbox permanently:

```
cd /var/www/html/M-ERP-Desktop-electron
sudo chown root:root node_modules/electron/dist/chrome-sandbox
sudo chmod 4755 node_modules/electron/dist/chrome-sandbox

npx electron --version
```

> Avoid **sudo npm install** because it can make the whole node_modules tree root-owned.

5. package.json

```
{
  "name": "m-erp-desktop-electron",
  "version": "1.0.0",
  "description": "M-ERP Desktop Application",
  "main": "main.js",
  "scripts": {
    "start": "electron ."
  },
  "devDependencies": {
    "electron": "^VERSION"
  }
}
```

Replace VERSION with the version installed by npm. The exact Electron version should be pinned in package-lock.json for repeatable builds.

6. Secure main.js

```
const { app, BrowserWindow, shell, session } = require('electron');

const isDev = !app.isPackaged;
const APP_URL = isDev
    ? 'http://127.0.0.1:8000'
    : 'https://erp.example.com';

function createWindow() {
    const win = new BrowserWindow({
        width: 1400,
        height: 900,
        minWidth: 1000,
        minHeight: 650,
        show: false,
        webPreferences: {
```

```
            preload: __dirname + '/preload.js',
            nodeIntegration: false,
            contextIsolation: true,
            sandbox: true,
            webSecurity: true
        }
    });

    win.once('ready-to-show', () => win.show());

    win.webContents.setWindowOpenHandler(({ url }) => {
        // Open external links in the user's normal browser.
        if (!url.startsWith(APP_URL)) {
            shell.openExternal(url);
        }
        return { action: 'deny' };
    });

    win.webContents.on('will-navigate', (event, url) => {
        if (!url.startsWith(APP_URL)) {
            event.preventDefault();
        }
    });

    win.loadURL(APP_URL);

    if (isDev) {
        win.webContents.openDevTools();
    }
}

app.whenReady().then(() => {
    session.defaultSession.webRequest.onHeadersReceived(
        (details, callback) => {
            callback({
                responseHeaders: {
                    ...details.responseHeaders,
                    'Content-Security-Policy': [
                        "default-src 'self' http://127.0.0.1:8000 https://erp.example.com; " +
                        "script-src 'self' 'unsafe-inline' 'unsafe-eval' http://127.0.0.1:8000 https://erp.example.com; "
                        "style-src 'self' 'unsafe-inline' http://127.0.0.1:8000 https://erp.example.com; " +
                        "img-src 'self' data: blob: http://127.0.0.1:8000 https://erp.example.com; " +
                        "font-src 'self' data: http://127.0.0.1:8000 https://erp.example.com; " +
                        "connect-src 'self' http://127.0.0.1:8000 https://erp.example.com"
                    ]
                }
            });
        }
    );

    createWindow();

    app.on('activate', () => {
        if (BrowserWindow.getAllWindows().length === 0) {
            createWindow();
        }
    });
});

app.on('window-all-closed', () => {
    if (process.platform !== 'darwin') app.quit();
});
```

Important: the CSP above is a starting point. Your Laravel Vite assets, CDNs, fonts, image hosts, AJAX endpoints, and third-party services may require additional approved origins. Do not copy a permissive CSP into production without testing.

7. preload.js

```
const { contextBridge } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
    platform: process.platform,
```

```
    isElectron: true
});
```

Because the existing ERP does not need Node APIs in its page, keep the preload bridge minimal. Do not expose raw ipcRenderer, filesystem access, shell access, or arbitrary command execution to the Laravel page.

8. Start the complete development setup

```
# Terminal 1 — Laravel
cd /var/www/html/M-ERP-Desktop
php artisan serve

# Terminal 2 — Electron
cd /var/www/html/M-ERP-Desktop-electron
npm start
```

Expected flow:

```
Laravel
http://127.0.0.1:8000
       ↓
Electron BrowserWindow
       ↓
Desktop ERP window
```

9. Test your existing Laravel ERP inside Electron

• Authentication: login, logout, session timeout and CSRF.

• Routing: every Blade route and redirect.

• AJAX/fetch: forms, Select2, Tabulator and API calls.

• Assets: Vite/CSS/JS, images, fonts and icons.

• File operations: upload, download, PDF, Excel and print.

• Reports and large tables.

• Browser-dependent features such as popups, downloads and external links.

• Remote MySQL operations through Laravel.

10. Laravel-side changes

In the normal online architecture, you usually do not need to rewrite controllers, services, repositories, Blade templates, Bootstrap, jQuery, Select2, or Tabulator merely because the UI is displayed by Electron.

Production Laravel requirements:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://erp.example.com

DB_CONNECTION=mysql
DB_HOST=remote-db-host
DB_PORT=3306
DB_DATABASE=erp_database
DB_USERNAME=erp_user
DB_PASSWORD=strong-password
```

> Never put the remote MySQL username/password into Electron. Electron only talks to Laravel over HTTPS. Laravel talks to MySQL.

11. Vite and asset URLs

If the Laravel application uses Vite, build the production assets normally on the Laravel server/deployment pipeline. Do not hard-code localhost asset URLs into production. Verify that CSS, JavaScript, fonts and images load from the intended HTTPS origin.

```
npm install
npm run build
```

Use the exact npm scripts already defined by your Laravel project's package.json if they differ.

12. Remote MySQL architecture

```
Windows PC
   ■
   ▼
M-ERP Desktop.exe
   ■
   ▼
Electron
   ■ HTTPS
   ▼
Laravel application server
   ■
   ▼
Remote MySQL
```

> Security rule: never expose MySQL directly to the desktop application. Restrict database access to the Laravel server/network where possible, use a dedicated database user with only required privileges, use TLS where appropriate, and keep database credentials server-side.

13. Common errors and fixes

Error / symptom	Likely cause	Fix
Missing script: start	package.json has no start script	Add "start": "electron ." and run npm start.
Electron command not found	Electron not installed locally	npm install --save-dev electron
chrome-sandbox permission error	Ubuntu SUID helper permissions	chown root:root chrome-sandbox; chmod 4755 chrome-sandbox
Laravel page does not load	Laravel server not running / wrong URL	Run php artisan serve and verify URL in Chrome.
ERR_CONNECTION_REFUSED	No server listening on configured port	Check php artisan serve and the configured host/port.
CSS/JS missing	Wrong Vite/asset origin or CSP	Check DevTools Network and Laravel Vite production setup.
AJAX fails	Wrong API URL, CORS, CSP, or HTTPS issue	Inspect Network/Console; use same Laravel origin where possible.
External links open inside app	Navigation not restricted	Use setWindowOpenHandler and will-navigate restrictions.
Blank window	JS error, CSP issue, or bad URL	Open DevTools in development and inspect Console/Network.
MySQL connection error	Remote DB/network/credentials	Test Laravel DB config; never move DB credentials into Electron.
composer.lock mismatch	composer.json changed without lock update	Update the specific missing package and commit composer.json + cor
npm/node_modules permission errors	npm was run with sudo	Fix ownership; avoid sudo npm install.

14. Debugging checklist

```
# Check Laravel
php artisan serve

# Check Node/npm
node -v
npm -v

# Check Electron
npx electron --version
```

```
# Start Electron
npm start

# Check package files
cat package.json
ls -la

# Check Electron processes
ps aux | grep electron
```

In Electron development, open DevTools and inspect Console and Network. A page that works in Chrome but fails in Electron is often revealing a URL, CSP, popup, download, certificate, or browser-environment difference.

15. Downloads, print and popups

ERP applications frequently use window.open(), print dialogs, generated PDFs, or file downloads. Test these explicitly. If the ERP uses a new window for printing or reports, implement a controlled window policy instead of allowing arbitrary new windows.

16. HTTPS and certificates

> Production should use valid HTTPS certificates. Do not add code that blindly ignores certificate errors. A certificate bypass can turn a desktop ERP into an easy man-in-the-middle target.

17. Electron security rules

Rule	Recommendation
Node integration	Keep nodeIntegration: false for Laravel content.
Context isolation	Keep contextIsolation: true.
Sandbox	Keep sandbox: true where compatible.
Web security	Do not disable webSecurity.
Navigation	Allow only your intended ERP origin.
New windows	Block by default and explicitly handle safe external URLs.
IPC	Expose narrow functions through contextBridge; validate sender and inputs.
Secrets	No database passwords, private API keys, or signing secrets in renderer/Electron source.
Dependencies	Keep Electron and npm dependencies updated and audit them.
CSP	Use a restrictive production CSP appropriate for the actual ERP assets.

18. Packaging into Windows EXE

After the ERP is stable inside Electron, use a packaging tool. Electron Forge is a practical official ecosystem option for importing an existing Electron project and generating distributables.

```
cd /var/www/html/M-ERP-Desktop-electron

npm install --save-dev @electron-forge/cli
npx electron-forge import

npm run make
```

The generated distributables are placed under the project's **out/** directory. The exact installer format depends on the platform and Forge configuration.

19. Windows build strategy

For reliable Windows distribution, build and test on Windows or use a controlled CI/build environment that supports the Windows target. Do not assume that an Ubuntu development build is identical to a production Windows installer.

Recommended release flow:

```
Development on Ubuntu
      ↓
Test Laravel
      ↓
Test Electron
      ↓
Commit package-lock.json
      ↓
Build Windows distributable
      ↓
Install on clean Windows machine
      ↓
Test login / reports / files / printing
      ↓
Code sign installer/app
      ↓
Release
```

20. EXE size and deployment

Electron packages Chromium and Node.js, so the desktop installer is substantially larger than a small native wrapper. This is expected. Your remote-Laravel architecture avoids bundling PHP and MySQL into the desktop installer, which greatly simplifies deployment.

21. Application updates

Keep Laravel deployment and Electron releases separate. Laravel fixes can normally be deployed to the server without rebuilding the desktop shell. Electron changes require a new desktop build. For production, choose an update mechanism and signing/release process before distributing to many users.

22. Git/repository strategy

```
Laravel repository
    ■■■ Laravel source, composer.json, composer.lock

Electron repository
    ■■■ main.js, preload.js, package.json, package-lock.json
```

Do not commit node_modules. Do commit package-lock.json. Do not commit .env files containing production secrets.

23. Production configuration

A robust implementation should avoid editing source code every time the server URL changes. A simple first version can use a production constant, but a mature release can use a signed/configured environment or deployment configuration. Never allow an untrusted remote user to change the ERP origin arbitrarily.

24. Optional: local-development URL vs production URL

```
const APP_URL = app.isPackaged
    ? 'https://erp.example.com'
    : 'http://127.0.0.1:8000';
```

This keeps Ubuntu development pointed to your local Laravel server while the packaged application points to the real HTTPS Laravel server.

25. What you do NOT need to change

Existing Laravel component	Normally keep it?
Blade templates	Yes
Bootstrap	Yes
jQuery	Yes
Select2	Yes
Tabulator	Yes
Laravel controllers	Yes
Services / repositories	Yes
Eloquent models	Yes
MySQL access code	Yes — keep it server-side
Electron-specific APIs in Blade	Only if you need desktop features

26. When you actually need Electron APIs

Only add Electron IPC/preload functionality for desktop-only requirements such as native menus, controlled file-system operations, OS notifications, printer integration, application-level shortcuts, or other capabilities that the browser cannot safely provide. Keep each capability narrow and validate inputs.

27. Final project structure

```
/var/www/html/
■
■■■ M-ERP-Desktop/
■   ■■■ app/
■   ■■■ bootstrap/
■   ■■■ config/
■   ■■■ database/
■   ■■■ public/
■   ■■■ resources/
■   ■■■ routes/
■   ■■■ storage/
■   ■■■ composer.json
■   ■■■ composer.lock
■
■■■ M-ERP-Desktop-electron/
    ■■■ main.js
    ■■■ preload.js
    ■■■ package.json
    ■■■ package-lock.json
    ■■■ node_modules/
    ■■■ out/                  # after packaging
```

28. Complete implementation checklist

■ Laravel works without Docker.

■ Remote MySQL works through Laravel.

■ Electron is in a separate directory/repository.

■ Electron starts with npm start.

■ main.js loads the Laravel URL.

■ nodeIntegration is disabled.

■ contextIsolation is enabled.

■ sandbox is enabled.

■ Navigation is restricted.

■ New windows/external links are controlled.

■ Production uses HTTPS.

■ No DB credentials or private secrets are stored in Electron.

■ Blade/Bootstrap/jQuery/Select2/Tabulator features are tested.

■ AJAX/API calls are tested.

■ Downloads, PDF, Excel and print are tested.

■ Laravel production assets are built correctly.

■ Windows packaging works.

■ The installer is tested on a clean Windows PC.

■ Application is code-signed before broad distribution.

■ Update/release strategy is documented.

29. Quick command reference

```
# Laravel
cd /var/www/html/M-ERP-Desktop
composer install
php artisan serve

# Electron
cd /var/www/html/M-ERP-Desktop-electron
npm install
npx electron --version
npm start

# Package
npm install --save-dev @electron-forge/cli
npx electron-forge import
npm run make
```

30. Recommended order for your project

```
PHASE 1
Laravel works without Docker
        ↓
PHASE 2
Remote MySQL works
        ↓
PHASE 3
Electron loads Laravel locally
        ↓
PHASE 4
Test all ERP features
        ↓
PHASE 5
Security hardening
        ↓
PHASE 6
Production HTTPS Laravel
        ↓
PHASE 7
Electron production URL
        ↓
PHASE 8
Windows packaging
        ↓
PHASE 9
```

```
Clean-machine testing
        ↓
PHASE 10
Code signing + release
```

31. Final recommendation for your ERP

For your existing Laravel ERP, the most maintainable design is to keep Laravel and Electron as two separate projects. Laravel remains responsible for business logic, authentication, authorization, database access, reports and web UI. Electron provides the desktop window and only adds native desktop capabilities when required.

> With remote MySQL, the production path should be **Electron** → **HTTPS** → **Laravel** → **Remote MySQL**. Do not put MySQL credentials in Electron and do not make the desktop app connect directly to port 3306.

32. Important limitations

This guide assumes your Laravel ERP is designed to work as an online web application. If you later require fully offline ERP operation, that is a different architecture involving local services/data synchronization and conflict handling. Do not mix that design into the current online Electron wrapper.

Reference notes

Electron's security guidance emphasizes current Electron versions, secure content, disabled Node integration for remote content, context isolation, sandboxing, restrictive navigation and CSP, safe IPC, and careful dependency management. Electron's packaging/distribution documentation describes packaging and distribution workflows; Electron Forge provides tooling for packaging an Electron application.

End of document